

TDD

“El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores...El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo”

Leyes TDD

- No escribir una línea de código hasta que no hayas escrito antes un test case que falla
- No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.
- No vas a escribir más código en Producción que el necesario para que el test pase.

Aplicar estas leyes no significa no aplicar test, el test se aplica siempre...

Escribir test

Hacer que test compile

que falle

Cambio para que el test pase

Corre el test para que pase

Quitar duplicaciones (refactor)

Patrones de TDD

(Tengo fotos en el celular)

utilizar la palabra test

armar sets de test

primero escribo el resultado de la prueba, despues la prueba

datos claros para la prueba

refactorizar

(imagenes en el celular)

concepto en general:

Sobre codigo ya escrito. no corrijo, sino, que emprolijo el codigo, solo que sea mas facil de leer, mantener, cumpla prinncipios de diseño. quito duplicidad de codigo.

como hacer refactoring

(cel)

asegurarse que las pruebas pasen
encontrar un código que huela mal
determinar como simplificar el código
hacer las simplificaciones

...

Resumen



ver video

test driven development

aprende a triangular en tdd

<https://www.youtube.com/watch?v=1gttkO9JKtU>

practico TDD

Vemos el ejercicio de practico taxi mobile

ver los assert, es una de las partes mas importantes.

1. Precondiciones
2. Pasos del caso de prueba
3. Resultados
 - assert
 - assert all/any
4. message final

TDD es prueba unitaria

ver diferencia con ATDD

03/10

Clase con Meles

Pruebas de Sw/Testing

PUD:

- Requerimientos
- Análisis
- Diseño
- Implementación
- **Prueba**
- Despliegue

Identificar defectos, un testing exitoso es aquel que identifique defectos. El software es una actividad humana intensiva, entonces, errores habrá.

No se puede asegurar calidad con testing. El testing evidencia defectos, cuya presencia se asume.

Desde que aparecen los frameworks ágiles, el agilismo intenta mostrar esta visión del testing, que es una actividad destructiva, que vienen a criticar el trabajo.

testing, actividad intenta buscar defectos, que su presencia se asume, el testing exitoso es el que lo encuentra.

Antes se pensaba que el testing se podía hacer cuando ya teníamos algo más o menos completo para empezar a probar. Esta idea esta basada de que si vos, los requerimientos tienen dos características, ser objetivos, y verificables. Objetivo: no ambiguo (que sea bonito), verificable, demostrar que esa característica se cumple. La característica de verificabilidad engancha a la prueba. si no tenemos una manera para poder mostrar que algo funciona, entonces no sirve...

Los enfoques empiezan a evolucionar a partir de esto, que se trata de adelantar el testing a partir del principio, ejemplo las user stories, relacionado con las 3c, la confirmacion, verificar escenarios de prueba que muestren al user owner que esa user story esta bien...

El testing es muy caro, por eso empresas deciden ni hacer testing, ejemplo Microsoft.

Tiempos/ Esfuerzos

- Requerimientos
- Análisis
- Diseño
- Implementación 33 - 35%

- **Prueba** 30 - 50%
- Despliegue

El primer modelo que se formaliza a veces es la implementación, todo lo demás no... En algunas empresas.

Mientras más exitoso es el proceso de software, a la gente de testing se le hará más difícil.

Hay que invertir en un desarrollo exitoso, para no gastar tanto en las pruebas, en testing :p

Terminología

Error vs defecto

Encuentra defectos, porque los defectos los introduce alguien de una etapa anterior

Cuando una acción se trasladó a una etapa posterior, es un **defecto**

Cuando se encuentra en una etapa, y en esa etapa se corrige, es un **error**

Mientras más viejo es, más costoso corregirlo.

Los errores hay que evitar que se conviertan en defecto

Tipos de errores (creo que algunos)

- Errores de arquitectura
- Errores de requerimiento

Falla

Un mal funcionamiento del sistema. A veces los errores y los defectos producen fallas en el sistema. Lo provoca un error o defecto.

Falla en el sistema

Mal funcionamiento, o que el sistema deje de andar, hay que ver la funcionalidad.

Si lo vemos con el sentido más amplio, en términos de satisfacción del cliente, un mal cálculo del IVA es una **falla**

Automatización del testing

Los perfiles profesionales de la gente de testing, hace que se haya especializado un poco más. Ya no cualquiera maneja esto, algo de implementación deben saber para usar una herramienta, se ganan un poco de reconocimiento en la industria.

En las empresas tiene la sensación de que si no programas no estas trabajando.

✓ Éxito

Testing es una actividad de control, hago una comparación contra algo, si tengo una versión de producto, la tengo que probar contra algo, contra los

requerimientos.

Un error que no se puede reproducir, no es un error

✓ Éxito

Requerimiento es general, caso de prueba específico

Caso de uso tiene sumatoria de escenarios, tenemos una plantilla que describía un escenario en particular. Esto es genérico. Se pide que seleccione país, el ingresa el país, no dice el VALOR. Cuando generamos un *caso de prueba*, es un caso concreto y específico. Basado en los requerimientos. Juan Perez está loguado, selecciona el país argentina, provincia cordoba, damos de alta el barrio nueva cordoba, es todo ESPECÍFICO

Testing metodológico

Planificación y diseño

--- 50%

Ejecución

Seguimiento y cierre

Se documenta el paso a paso de cómo ocurrió el error

Procesos de testing:

- Planificación y diseño
 - Artefacto *Plan de Prueba* y el más importante, *Caso de Prueba*, eso me determina que hago un Testing Metodológico y si lo puedo reproducir. Se muestra un caso de prueba específico de pasos...
 - Caso de Prueba tiene *seteo y escenarios* y *resultado esperado*. El seteo es específico. El caso de prueba es el pasa...
 - *Se requiere el nivel de detalle más fino*
- Ejecución
 - Reporte de defectos
 - Requiero lo anterior para hacer los casos de prueba.
 - Evidencia de acción de prueba
- Seguimiento y cierre (Informa la situación de cómo se halló)

Testing AdHoc

Prueba manual?

No se puede reproducir.

No sirve para nada

Sentarme a ver cosas :p

Si pongo letras en la fecha?

Ponele como fuerza bruta

No tengo algo para solventar
Si salta algo, como lo puedo comunicar
Informar paso a paso de como se reproduce

Ciclo de prueba

Un ciclo de ejecución y seguimiento y cierre.
Planificación y diseño no

Ciclo cero

Primer ciclo que se hace

Invalidante

No puedo hacer nada en el sistema

Adelantar

La parte del 50 arriba se puede adelantar, cuando tenemos ya los requerimientos acordados...

Lo de abajo no lo puedo adelantar, lo de arriba si se puede, se debe adelantar...

Adelantamos el plan de prueba, tomamos como entrada los requerimientos y hacemos eso...

Prueba tiene

Validación

Determina adecuación, si se definió , se creó el producto correcto

Verificación

Que funcione correctamente

Determina corrección, si eso que hice, funciona bien

Advertencia

No hay una definición de requerimientos, por eso en muchas empresas es difícil implementar pruebas

Error que se comete mucho cuando no tengo requerimientos, y lo unico que tengo es el código, puedo hacer verificación, pero no validación...

Los casos de prueba tienen que estar basados en los requerimientos.

No hay que esperar a que esté terminado el código para probar..

Hay que adelantar el proceso de testing...

Niveles de Prueba

4 niveles de prueba

Se prueba de lo particular a lo general

- Prueba unitaria - lo hace el desarrollador
- Prueba de integración - Prueba de Interfaces - Que ensamble
- Prueba de sistema o versión
- Prueba de aceptación de usuario

Ambientes de prueba / Entorno

- Desarrollo
- Prueba - stage
- Preproducción
- Producción

CHATGPT

¿Qué es el Testing?

El **testing** es el proceso de **probar un software** para verificar que **funcione correctamente**, que **haga lo que se espera** y que **no tenga errores**.

"El testing es una **disciplina del proceso de desarrollo de software** cuya finalidad es verificar y validar que un producto cumpla con los **requisitos especificados**, detectando errores, fallos o desviaciones respecto al comportamiento esperado."

-  **Verificar** que el software hace lo que tiene que hacer (según los requerimientos).
-  **Detectar defectos** antes de que lleguen al usuario final.
-  **Prevenir errores futuros**, entendiendo cómo se comporta el sistema en distintos contextos.
-  **Mejorar la calidad** del producto final (robustez, confiabilidad, usabilidad).
-  **Asegurar la conformidad** con criterios legales, normativos o contractuales (según el tipo de software).

El objetivo principal del testing **no es demostrar que el software funciona**, sino **encontrar los errores que podrían impedir que funcione correctamente**.